CSCE 312 LAB 4

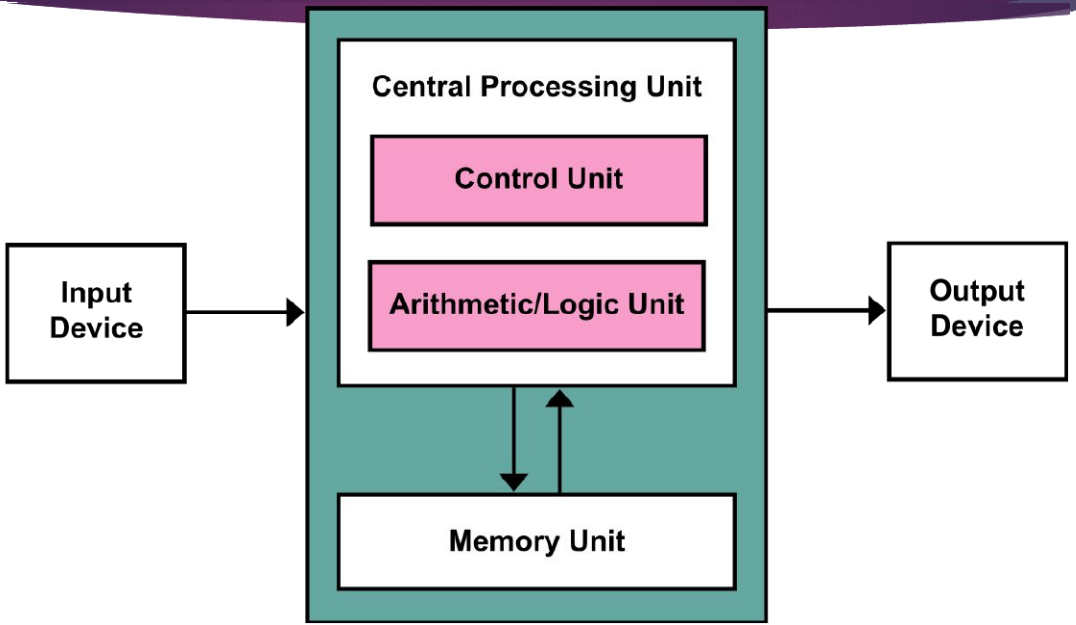
LAB4

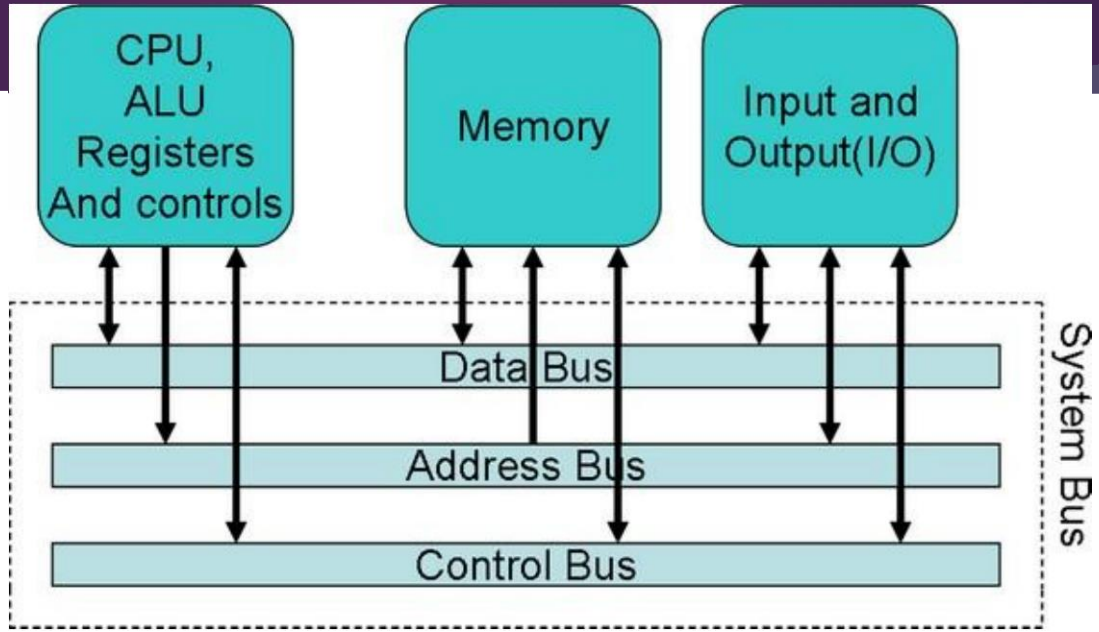
- Report Deadline: 11 October midnight
- Demonstrations: 12 October Labs
- To be done in groups of 4
- You need to submit a common lab report **as a group** along with all the files you create
- You also need to submit an additional 1-page report **individually** to explain what your individual role and contribution was and critique the group design process.

PROBLEM 1

- Start building a processor
- For this lab, you design and simulate the peripheral components (IO interface circuitry, address/data/control bus, encoders, decoders, counters, registers)
- First, let us understand how a processor works

Von Neumann Architecture




CPU,
ALU
Registers
And controls
Memory
Input and
Output(I/O)
Data Bus
Address Bus
Control Bus
System Bus

COMPONENTS

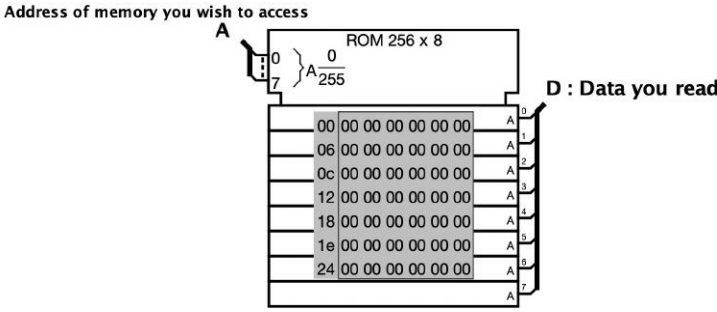
- INPUT: Mouse, Keyboard, etc
- OUTPUT: Speaker, Monitor, Printer, etc
- MEMORY: DRAM, SRAM, etc
- CPU: Central Processing Unit
  - ALU: Arithmetic Logical Unit (Adders, Subtractors, Comparators, etc)
  - Control Unit: Send Signals (Get data, Send data, Read Data, Write Data)
  - Registers: Temporary storage (data just retrieved from memory, data that is being calculated)
  - Memory?

BUSES

- CONTROL BUS: The common signals that are transferred to the control bus from CPU to devices and vice versa are memory read, memory writes, I/O read, I/O write. (what to do?)
- ADDRESS BUS: The address bus is used by the CPU to send the address of the memory location or the input/output port that is to be accessed at the instant. (where to get data from/make changes?)
- DATA BUS: A data bus is used to carry the data and instructions from the CPU to memory and peripheral devices and vice versa. (what to send/get?)

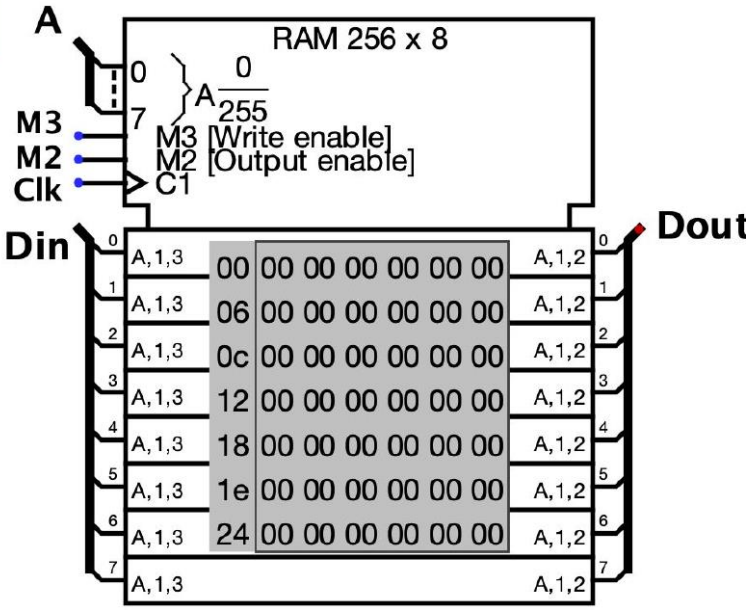
ROM in Logisim

- Read Only Memory, means can only access the data not change it.
- A: Address of the memory you try to access.
- D: Data you retrieve present at A
- A takes 8 bits
  - Means you can address $2^8 = 256$ things
- D gives 8 bits
  - Means each address corresponds to 1 byte of data
- Therefore, you have 256 x 1 B = 256B of data inside the ROM



RAM in Logisim

- Random Access Memory
- Volatile Memory (lose it on turning power off)
- can be read and changed in any order, typically used to store working data and machine code.
- A: Address to write to/read from
- M3: writing (store)
- M2: reading (load)
- Clk: Clock
- Din: Data you write
- Dout: Data you read
- A: 8 bits meaning 256 things
- D: 8 bits meaning each ‘thing’ is 1 B
- So, total we have 256x1B = 256B data in the RAM



What you need to do:

- read from ROM
  - Control Bus: read from ROM
  - Address Bus: where to read
  - ROM: read the data
  - Data Bus: put the read data in the data bus
- read from RAM
- write to RAM
- read from I/O device
- write to I/O device

How to implement

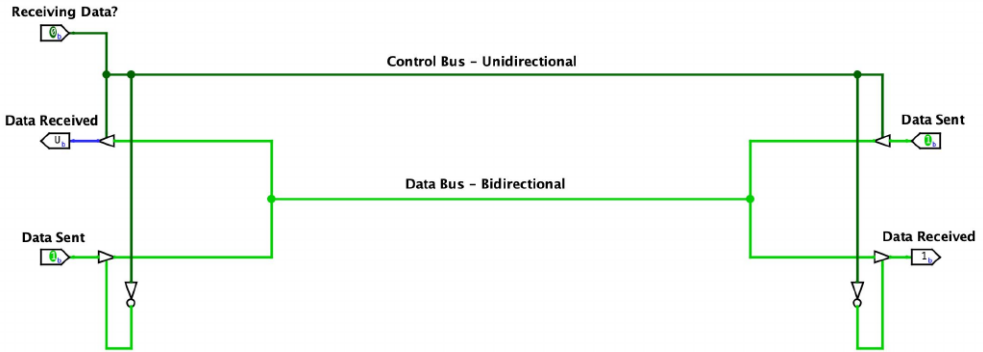
1. Assume that you have the Control, Address and Data signals from CPU.
2. Use these to create the three buses. Data Bus needs to be bidirectional.
3. Build a Memory Unit (consisting of RAM and ROM)
4. Build an I/O Unit
5. Use the control signals to figure out which of the 5 operations you need to do. (Build Decoder)
6. Depending on the output of decoder, send read/write enable, address and data(if required) to the Memory Unit or I/O Unit and get data from the unit back to the data bus (if required)
7. Whenever you try to write data or read data, you need to store that data in a Register (called the Data Buffer) so that you can use this stored data again if you need it in the next operation.

1. CPU

- Add a Control Input
- Add an Address Input
- Add a Data Input

2. Create Buses

- For unidirectional bus just directly use the input you have
- For bidirectional bus, implement using controlled buffer
  - You want to use the same bus to send data when writing and retrieve data when reading
  - How to make such a bus?



3. Build a Memory Unit

- Include a RAM and a ROM in the Unit
- Using the control signals, figure out which one you need for the current operation. Then, depending on if it is a read (address only) or a write (address and data) do the operation (write it inside the RAM or get the read data from RAM/ROM back to the data bus)

4. Build an I/O Unit

- The I/O Unit has 4 input devices and 4 output devices.
- You take 8 bit input data using the input devices and you give back 8 bit data output using the output devices.
- How to address the I/O devices is up to you.
- How to implement?
  - You take four 8 bit inputs from the input devices and send it to the CPU
  - You have four 8 bit output devices (Lab 3 Problem 3) which take data from your CPU
  - If your operation is to read from IO then you choose the correct Input device using the address and read data from that input and send it to the data buffer.
  - If your operation is to write to the IO then you find the correct Output Device using the address and write to it the data you obtain from the CPU.

5. Decoder

- You will input the control signals to figure out which of the required 5 operations are to be done and hence whether RAM, ROM, I/O is to be used and if we need to read/write.

6. Connections from Buses to Units

- If it is a read, send address of location to the appropriate unit and get back read data from that unit.
- If it is a write, send address and data to the appropriate unit.
- Make sure every time you try to read a data, you store it in the data buffer and every time you try to write some data, that data is stored in the buffer as well.

7. Data Buffer

- Implement using Register in Logisim.
- If current operation requires you to write, then first input the data to this buffer and then use the output of this buffer to send data through data bus to the units.
- If current operation requires you to read, then finish the read operation from unit and then input the data you just acquired into the data buffer.

REPORT:

- Mention how the control signals for the 5 operations are implemented.
- Mention how the addressing for the I/O devices is done
- Explain how tri state buffers using controlled buffer ensures that your bus is bidirectional and how the buffer is handled differently in case of different operations.
- Mention some examples for your 5 operations that work on your design and what are the inputs and outputs for each of those examples.
- Screenshots of whole design and all the subparts (Memory and I/O Units)

PROBLEM 2

For 8-bit inputs:

- Design an adder (2 input)
- Design a subtracter – Using 2's complement (2 input)
- Magnitude Comparator (2 input)
- Left and Right Barrel Shifter (2 inputs: The number and shift amount). *[Extra Credit]*

Refer to Lecture Slides to understand how to implement each of these.

PROBLEM 3 *(Extra Credit)*

- A single row of 20 LEDs
- One input
- If input is 0: The LEDS blink from Right to Left in sequence.
- If input is 1: The LEDS blink from Left to Right in sequence.
- Each LED should be ON for 0.25 seconds before it turns OFF and next LED turns ON.
- How to implement?
  - Finite State Machine with 20 states.
  - Rather than writing a very large truth table for each state transition and finding the complex Boolean logic, you can use a counter to increment state values one by one. You increment in case of input 0 and decrement in case of 1.
  - After implementing, change the decoder to ROM and implement the same thing

Files to be submitted

Group Report
Individual Report
All Logisim Files with proper names (Problem1,2,3)